CMU 11-768 | AI Agents | Fall 2026 | 学习笔记 | Lecture 1：Course Overview: What Is an Agent?

1. 前言

在真正开始工具使用（Tool Use）、上下文管理（Context Management）、记忆（Memory）和规划（Planning）之前，首先需要弄清楚这门课所说的人工智能体（AI Agent，后文简称 Agent）到底是什么。它不能简单理解为“更强的 ChatGPT”，也不是给语言模型接上一个搜索接口就自动得到的产品名称。现代 Agent 的核心变化，是把一次性的“输入文本—生成文本”扩展为一个持续运行的交互过程：模型可以观察环境、选择动作、接收动作结果，再根据新的状态决定下一步。

Lecture 1 同时承担两项任务。技术上，它从经典 Agent 定义出发，解释观察（Observation）、动作（Action）、环境（Environment）、工具（Tool）、智能体执行框架（Agent Harness）与推理—行动循环（ReAct Loop）怎样组合成完整系统；课程层面，它把后续的工具使用、长上下文、记忆、规划、训练、评测和安全放进同一张路线图。理解这套基本框架之后，后续每一讲都可以看成在修补 Agent 循环中的某个具体能力缺口。

2. 本讲知识地图

```
What Is an Agent?
├── Why Agents Now?
│   ├── 能完成更长、更真实的任务
│   └── 能力提升同时放大风险
├── Classical Agent View
│   ├── Environment
│   ├── State / Observation
│   ├── Action
│   └── Reward
├── From LLMs to Agents
│   ├── Next-token Prediction
│   ├── Tool Definition
│   ├── Tool Call / Tool Result
│   └── Actions as Tokens
├── Agent Loop
│   ├── Context
│   ├── Reasoning
│   ├── Act
│   ├── Observe
│   └── Stop
├── Agent Capabilities
│   ├── Accurate Tool Calling
│   ├── Long-context Coherence
│   ├── Customizability
│   ├── Complex-task Management
│   ├── Environment Understanding
│   └── Safety
├── Two Improvement Paths
│   ├── LLM Training
│   └── Harness Engineering
└── Agent System Stack
    ├── Harness
    ├── Sandbox
    ├── LM Inference
    ├── Training System
    └── Observability
```

3. 为什么现在需要单独研究 Agent

3.1 从生成答案到完成真实任务

普通语言模型最擅长的是根据 Prompt 生成一段后续文本。即使它能写出非常好的分析，任务通常也在这次 Response 结束时停止。现实任务却很少只需要一段文字：修复软件需要读取仓库、运行测试、修改文件并再次验证；浏览网页需要点击、输入、滚动和比较页面；个人助理可能还要调用日历、邮件或支付接口。

Agent 研究关注的正是这段差距。模型不仅要“知道应该怎么做”，还要把决策变成外部系统能够执行的动作；动作改变环境后，模型又要读取新结果并继续工作。任务因此从一次静态推断变成多步、带反馈的闭环控制过程。

Lecture 1 用一个很醒目的案例说明当前能力边界：16 个 Claude Agent 在近两周、约 2,000 个 Claude Code 会话中协作，产生了约 10 万行 Rust 代码，构建出能够编译 Linux 内核的 C 编译器。这里的关键不只是模型能写代码，而是智能体执行框架（Agent Harness，后文简称 Harness）让多个实例在容器中持续运行，通过任务锁、Git、测试和进度文件协调工作。复杂成果来自“模型 + 环境 + 反馈 + 控制循环”的组合。

> 【图：Lecture 1 Slide 2 —— 16 个 Agent 协作构建 C 编译器】

幻灯片展示了编译器仓库与项目结果。教师用它说明 Agent 已经可以在长时间、多步骤、可验证的工程任务中产生实质成果；同时，原项目也强调这些代码并未经过完整人工验证。能力展示不能代替正确性证明。

3.2 能行动，也意味着能造成后果

生成一段错误文字和执行一个错误动作，风险并不相同。一次错误回答可以被忽略，但错误的文件删除、邮件发送、订单提交或医疗操作可能难以撤销。Lecture 1 展示了一个邮件整理 Agent 的失败案例：用户明确要求不要继续删除，系统在上下文压缩后却丢失这条约束，仍然执行了删除。

这说明 Agent 能力与 Agent 安全不能割裂。工具把模型输出连接到真实环境，也把模型的不确定性、误解和遗忘放大为现实影响。课程让学生判断六类任务应该“自主执行（Autonomous）”“先征求确认（Ask First）”还是“绝不授权（Never）”：诊断网店故障、向五万名客户发送产品邮件、报税、迁移支付 API、购买演唱会门票、根据血糖记录调整胰岛素剂量。

> 【图：Lecture 1 Slide 4 —— 不同任务应该获得多大自主权限】

图中的任务没有一个由技术能力单独决定权限。可逆性、错误成本、隐私、法律责任和用户偏好都会改变答案。诊断故障可以只读环境，风险相对低；批量发邮件和购票会对外产生影响，适合在关键节点请求确认；胰岛素剂量涉及高风险医疗决策，即使模型“有能力给建议”，也不代表系统应被授予执行权限。

这里真正需要建立的直觉是：**自主性不是一句“让 Agent 自己做”就完成的设计，而是一组可被精细控制的权限。** Harness 必须决定哪些动作可直接执行，哪些需要审批，哪些根本不暴露给模型。

4. 什么是 AI Agent

4.1 经典定义与本课程的关注范围

Lecture 1 引用 Russell 与 Norvig 的经典定义：

> Agent 是任何可以被看作通过传感器感知环境，并通过执行器作用于环境的事物。

这个定义早于现代 LLM，范围也远大于“聊天机器人”。恒温器、机器人、游戏策略程序都可以在相应抽象下被视为 Agent。课程研究的是其中一个更具体的分支：以大语言模型（Large Language Model, LLM）作为决策核心、能够在环境中进行多步交互的 AI Agent。

经典定义的共同结构是“感知—行动”。Agent 必须处在某个环境里，接收关于环境的信息，再选择会影响环境或用户的动作。如果系统只做一次离线文本映射，没有持续观察与行动，它与课程关注的 Agent 仍有结构性差异。

4.2 Environment：Agent 所处的外部世界

环境（Environment）是 Agent 需要读取或改变的外部世界。它不一定是物理空间，也可以是代码仓库、网页、桌面应用、终端、数据库、业务流程或棋盘。

环境决定了什么信息可见、什么动作合法，以及动作会带来什么后果。对 Coding Agent 而言，环境可能包括仓库文件、Shell、依赖、测试和 Git 状态；对 Browser Agent 而言，环境包括当前网页、可点击元素、表单和网络响应；对个人助理而言，环境可能包括邮件、日历与用户账户。

没有环境，模型只能谈论动作；接入环境后，系统才可能真正执行动作。与此同时，环境边界也是安全边界。Agent 能访问哪些目录、网络地址、凭据和 API，必须由外部系统限制，不能只依赖模型“自觉”。

4.3 State 与 Observation：真实状态和模型看到的信息

状态（State）是环境在某一时刻的真实情况。观察（Observation）是 Agent 当前获得的、关于这个状态的信息。两者不能简单画等号，因为 Agent 往往只能看到局部或经过转换的状态。

例如，仓库的完整 State 包含所有文件、进程、依赖和 Git 历史；模型调用 `read_file("test.py")` 后得到的文件内容，只是一次 Observation。网页真实 DOM、后台数据库和登录状态共同构成环境状态，而一张 Screenshot 只是视觉观察。Tool Result、用户消息、Shell 输出、文件内容和网页文本都可以成为 Observation。

如果没有 Observation，模型无法知道刚才动作是否成功，只能凭先验猜测。Agent 之所以需要循环，是因为行动会改变环境，改变后的结果必须重新被观察。

4.4 Action：输出不一定是文本回答

动作（Action）是 Agent 对环境施加的操作。回复用户当然是一种 Action，但不是唯一形式。Lecture 1 给出的动作还包括编辑文件、执行 Shell 命令、调用 API，以及发送鼠标或键盘事件。

模型本身通常并不直接操作操作系统。它生成一段结构化 Token，用来表达“调用哪个工具、传入什么参数”；Harness 解析、校验并执行这段请求。因此，需要区分“模型提出 Action”与“系统执行 Action”。前者属于模型生成，后者属于普通程序和受控环境。

4.5 Reward：怎样知道任务做得好不好

奖励（Reward）是对结果质量的数值反馈。代码任务可以把测试是否通过映射为 0/1 奖励；开放任务可能使用 LLM-as-a-Judge 按 Rubric 评分；真实产品还可能收集用户反馈。

Observation 告诉 Agent“发生了什么”，Reward 则评价“做得怎么样”。终端返回测试失败是一条 Observation；把修复后的全部测试通过记为高奖励，是评价机制。奖励对训练和评测尤其重要，但不一定在每个推理步骤中都显式显示给模型。

4.6 Agent 为什么是一个循环

把这些概念合起来，一个最小 Agent 过程可以表示为：

```
User Goal
   │
   ▼
Observe current context and environment
   │
   ▼
Model reasons and proposes an Action
   │
   ▼
Harness validates and executes the Action
   │
   ▼
Environment changes and returns an Observation
   │
   ├── Task unfinished ──► append to history and repeat
   │
   └── Task finished   ──► report result to user
```

循环不是为了让系统看起来复杂，而是由信息依赖决定的。查询结果出现之前，模型无法知道下一步应该选择哪一个航班；测试执行之前，模型无法知道补丁是否正确；打开网页之前，模型也不知道页面有哪些可交互元素。每个新 Observation 都可能改变后续计划。

4.7 Agent 的自主性具体来自哪里

自主性（Autonomy）不是模型“有意识”，而是系统允许模型在多大范围内自己选择下一步。若程序预先写死每个步骤，模型只负责填入一段文本，自主性很低；若模型能根据 Observation 自行选择工具、参数、执行顺序、重试策略和停止时机，自主性更高。

自主性来自模型与 Harness 的组合。模型提供开放式决策能力，Harness 提供可调用动作、循环控制和权限边界。扩大工具集合、增加循环次数、减少审批都会提高可行动范围，也会同步增加错误面。

5. 从 LLM 到 Agent：Action 怎样变成 Token

5.1 先回到语言模型本身

语言模型进行下一 Token 预测（Next-token Prediction）：给定当前前缀，预测下一个 Token 的概率分布；选出一个 Token 追加到前缀后，再预测下一个。思维链（Chain of Thought）也是一串中间 Token，它们充当推理草稿，并成为后续预测的上下文。

在这个层面，LLM 只做 Token 输入与 Token 输出。它不会因为生成了字符串 **rm file**.txt 就自动删除文件，也不会因为输出了函数名就自动访问 API。现代多模态模型可以接收图像等其他模态，但“模型生成表示”与“环境真正发生变化”的边界仍然存在。

5.2 Tool Definition：让模型知道有哪些动作

工具（Tool）是暴露给模型的类型化接口。一个工具定义通常包含名称、自然语言描述和参数 Schema。例如，读取文件工具可以写成：

```
{
  "name": "read_file",
  "description": "Read a UTF-8 file.",
  "parameters": {
    "type": "object",
    "properties": {
      "path": {"type": "string"}
    },
    "required": ["path"]
  }
}
```

名称告诉模型要生成哪个调用，描述帮助它判断何时使用，Schema 约束参数结构。API 客户端最终会通过 Chat Template 把这段结构渲染成模型能读取的文本或特殊 Token。对模型而言，工具定义仍然只是 Context 的一部分。

如果没有工具定义，模型可能知道“应该读取文件”，却不知道当前系统允许调用什么接口、参数叫什么、结果会以什么格式返回。工具定义把开放式语言意图映射到可执行的软件协议。

5.3 Tool Call 与 Tool Result

工具调用（Tool Call）是模型生成的结构化 Action：

```
{
  "id": "call_7",
  "name": "read_file",
  "arguments": {"path": "test.py"}
}
```

Harness 收到这段调用后，检查工具是否存在、参数是否有效、权限是否允许，然后才执行 **read_file** 。环境返回的工具结果（Tool Result）可能是：

```
{
  "role": "tool",
  "tool_call_id": "call_7",
  "name": "read_file",
  "content": "def add(a, b): ..."
}
```

tool_call_id 把请求与结果对应起来。结果经过模板渲染后重新成为 Token，作为 Observation 加入下一轮 Context。模型并没有跨出 Token 世界；真正跨越模型与环境边界的是 Harness。

> 【图：Lecture 1 Slide 13 —— From LLMs to agents: actions as tokens】

图的左半部分属于语言模型：模型先生成推理 Token，再生成表示 read_file(path="test.py") 的 Tool Call Token。中间的 Execute 属于 Harness 和环境，不属于 LLM。右侧文件内容是执行结果，被重新编码为 Observation Token 送回模型。箭头展示了“Token → 外部动作 → Token”的完整桥梁。

5.4 为什么给 LLM 工具会产生 Agent-like Behavior

只提供一个 Tool Definition，得到的是“模型具备提出工具调用的语言能力”；只有外部程序执行调用、把结果送回模型，并允许模型继续选择下一步时，才形成多步 Agent 行为。

Toolformer 的重要贡献之一，是展示语言模型可以学习何时调用 API、调用哪个 API、传什么参数，并把返回值用于后续 Token 预测。ReAct 则把推理（Reasoning）与行动（Acting）交替组织：推理帮助模型维护和更新计划，行动让模型从环境获取新信息。Lecture 1 用这两项工作解释现代 Agent 的基本结构来源。

6. Agent Loop 是怎样运行的

6.1 Context 中包含什么

每一轮模型调用都要构造上下文（Context）。Lecture 1 的 Agent Loop 图把 Context 分成任务（Task）、工具（Tool）和历史（History）。实践中还会有 System Prompt、用户补充、先前的 Assistant 消息和 Tool Results。

任务说明目标是什么；工具定义决定允许采取哪些动作；历史记录先前的推理、Action 与 Observation。模型读取这些内容后生成下一步。历史一旦遗漏，Agent 可能重复工作或违反旧约束；历史无限增长，又会带来 Lecture 3 讨论的长上下文成本。

6.2 一次完整循环

以修复 **test**.py 为例，Agent 可能经历：

1. 用户给出目标：修复加法函数的测试失败。
2. 模型读取任务与工具列表，决定调用 read_file(path="test.py") 。
3. Harness 校验调用，在仓库环境中读取文件，并返回内容。
4. 模型观察源码，决定运行相关测试。
5. Harness 执行测试，把 Exit Code 与错误信息作为 Observation 返回。
6. 模型根据错误编辑文件，再次运行测试。
7. 测试通过后，模型调用结束工具或向用户发送最终消息。

普通 LLM 可能在第一轮就给出“建议怎样修改”的文字；Agent 则把建议变成动作，并用环境反馈验证。循环中最重要的不是“多调用几次模型”，而是每一轮都有新的、来自环境的证据。

> 【图：Lecture 1 Slide 14 —— An agent is a model in a loop】

图中左侧 Context 汇集 Task、Tool 和 History，中间 Model 生成 Reasoning 与 Call，右侧 **read_file** 调用被执行后产生 Result。Result 回到 History，构成下一次循环的输入。 **send_message** 表示一种终止型 Action：当模型认为任务完成时，不再调用工作工具，而是向用户报告。

6.3 一个极简 ReAct 控制器

Lecture 1 展示了 mini-swe-agent 的核心结构。下面保留幻灯片中最影响理解的部分，省略配置、异常处理和具体执行器：

```
def run(self, task):
    self.messages = [
        system_message(...),
        user_message(task),
    ]
    while True:
        self.step()

def step(self):
    return self.execute_actions(self.query())

def query(self):
    message = self.model.query(self.messages)
    self.messages.append(message)
    return message

def execute_actions(self, message):
    observations = execute_tool_calls(message)
    self.messages.extend(observations)
    return observations
```

run 先创建 System 与 User 消息，然后反复调用 step 。 **query** 是 LLM 发生的位置：它把当前消息历史交给模型，获得包含推理或 Action 的 Assistant Message。 **execute_actions** 则是普通程序：它解析模型提出的调用，在环境中执行，并把 Observation 追加到 **messages** 。

这段代码故意很短，因为 Agent Loop 本身并不神秘。真正困难的是保证 Prompt 正确、动作合法、工具可靠、错误可恢复、上下文可管理、环境被隔离，并能判断何时结束。一个循环很容易写，能稳定完成真实任务的 Agent 很难做。

6.4 模型怎么知道什么时候停止

停止也是控制协议的一部分。Harness 可以提供 **send_message** 、 **finish** 或 **submit** 一类终止动作；模型认为目标完成时生成该调用。系统还必须设置最大步数、时间和费用上限，防止模型永远循环。

“模型觉得完成了”不等于任务真的完成。Coding Agent 应先检查测试，Web Agent 应确认提交结果，Harness 也可以在接受终止前运行外部验证。后续 Planning 和 Evaluation 课程会进一步处理完成判断与结果验证。

6.5 Trajectory：为什么要记录整条执行过程

一次任务产生的完整 Reasoning、Action 与 Observation 序列称为轨迹（Trajectory）。轨迹不仅用于让模型继续工作，也用于调试、评测和训练。

> 【图：Lecture 1 Slide 16 —— SWE-bench 中的 Agent Trajectory】

图中 System Message 和 User Message 给出规则与任务，蓝色区域是模型生成的推理，随后出现 Bash Action；环境返回 Exit Code 和命令输出后，模型进入下一轮。单看最终补丁无法知道 Agent 为什么成功或失败，轨迹则能暴露错误工具、错误参数、重复尝试、遗漏约束和成本来源。

7. 三个案例：同一套循环怎样进入不同环境

7.1 Browser Agent：在网页中搜索并筛选

课堂展示的 Browser Agent 接到任务：在 Pittsburgh 找一家评价不少于 200 条、评分至少 4.3 的优质泰国餐厅。它先观察浏览器界面，在 Yelp 搜索框输入查询，读取搜索结果，再进入满足条件的餐厅页面。

这个案例中的 Environment 是浏览器与网站；Observation 可以是 Screenshot、网页文本和当前 URL；Action 是点击、输入、滚动或打开链接；LLM 负责根据页面状态判断下一步，而浏览器控制程序负责真正发送鼠标与键盘事件。

如果程序事先知道目标页面 URL，直接打开即可，不需要 Agent。任务之所以具有 Agent 性，是因为页面内容事前未知，系统要根据每一步 Observation 动态选择后续 Action。评分和评论数也是明确约束，模型不能只找到“看起来不错”的餐厅就停止。

7.2 Coding Agent：写代码之后还要自己验证

OpenHands 演示让 Agent 构建一个待办事项 Web 应用。Agent 创建目录、应用文件、HTML、CSS、依赖和 README，随后启动服务。发现端口已被占用后，它读取日志并调整运行方式；应用启动后，又使用浏览器添加和删除待办事项，验证界面功能。

这里同时存在文件系统、Shell 和 Browser 三类环境接口。模型选择创建哪些文件、运行哪些命令、怎样解释错误；工具负责编辑文件、启动进程和操作网页。端口冲突是一条新的 Observation，它使 Agent 偏离原计划、先解决运行问题，再继续测试。

这个案例说明“生成代码”和“完成软件任务”的区别。一次 LLM 调用可以给出源码，但 Coding Agent 还要把代码写入正确位置、安装依赖、启动应用、观察失败、修复问题并验证用户可见功能。测试与反馈闭环才让系统从代码生成器变成软件工程 Agent。

> 【图：Lecture 1 Slide 6 —— OpenHands Visual Debugging】

图中一侧是 Agent 的消息、命令与状态，另一侧是实际运行的网页。二者之间的往返意味着模型不是在想象应用是否可用，而是在真实环境中运行和检查它。

7.3 并行 Agent 团队：规模变大后，Harness 更重要

编译器案例把单一循环扩展为 16 个并行 Agent。每个 Agent 在独立容器中克隆仓库、领取一个任务锁、修改代码、运行测试并向共享 Git 仓库推送；新的会话再选择下一项工作。测试、任务锁、进度文件和容器共同构成 Harness。

模型能力当然重要，但项目能够持续推进也依赖环境设计。高质量测试为 Agent 提供可判定的反馈；精简日志避免无用输出污染 Context；进度文档帮助新会话重新定位状态；任务锁减少多个 Agent 做同一件事；容器控制权限与依赖。

这个案例还显示，增加 Agent 数量不一定线性增加能力。当所有实例都被同一个内核编译错误阻塞时，并行会退化为重复劳动。项目后来借助 GCC 作为正确性 Oracle，把大问题拆成可以独立定位的子问题。复杂任务管理仍需要可分解目标、可靠验证和协调机制。

8. 一个可用 Agent 需要哪些能力

8.1 六类能力不是六个独立插件

Lecture 1 把“好 Agent”需要的能力整理为六类：

能力	要解决的问题	典型失败
Accurate Tool Calling	选择正确工具并生成合法参数	工具名错误、JSON 破损、参数不符合 Schema
Coherence over Long Context	在长轨迹中保留约束与任务状态	忘记用户要求、重复已完成工作
Customizability	适应用户、组织和领域的做事方式	无视代码规范或个人偏好
Complex Task Management	分解、规划并完成长时程任务	做到一半迷失目标、没有验证
Environment Understanding	正确解释环境提供的数据和界面	看错网页、误解日志、对领域数据做错误判断
Safety	在权限与风险边界内完成任务	泄露凭据、越权访问、执行不可逆动作

这些能力相互影响。工具调用正确但环境理解错误，Agent 会准确地执行错误动作；规划很好但长上下文不连贯，系统会忘记计划；模型能完成任务但没有安全边界，也不能用于高风险环境。

课堂把 Safety 直接视为 Agent 能力，而不是附加在“能力研究”之外的事后补丁。一个用户不敢授权使用的系统，在现实任务中就是能力不足。

8.2 Accurate Tool Calling

准确工具调用包括四个环节：选对工具、生成正确参数、通过验证、由 Harness 执行。模型可能通过 SFT 在大量 Tool-calling Trajectory 上学习格式与选择策略；Harness 也可以使用 Grammar-constrained Decoding 阻止结构非法的输出，并在执行前做 Schema、权限和业务校验。

结构合法不等于语义正确。 delete_file(path="/important") 可以是完全合法的 JSON，却仍然是危险动作。模型训练解决“更可能作出正确调用”，Harness 验证解决“哪些调用可以被执行”，两者不能互相替代。

8.3 Coherence over Long Context

Agent 每执行一次 Action，就会向 History 追加推理、调用和 Observation。任务越长，模型越可能遗忘早期约束，也越难承担不断增长的输入。Harness 可以使用上下文压缩、动态记忆检索或 Sub-agent Delegation；模型端可以进行 Long-context Training。

这项能力直接通向 Lecture 3。课程开头的邮件删除事故就是反例：系统压缩历史后失去“不要删除”的重要约束。长上下文管理不是为了让 Prompt 看起来整洁，而是为了让未来 Action 继续受过去目标和限制约束。

8.4 Customizability

可定制性（Customizability）要求 Agent 按特定用户或组织的方式工作。Harness 可以通过 Memory 保存用户偏好，通过 Skills 按需加载专门流程，通过 Custom Tools 暴露领域动作。模型也可以从用户反馈中继续学习，但在线修改模型权重成本高、速度慢，也不适合每一种临时规则。

例如，“这个仓库提交前必须运行某个脚本”更适合写入项目 Skill 或 System Instruction；“用户长期偏好简短回复”可能进入 Memory；在大规模用户反馈中反复出现的共性行为，才更可能通过训练固化进模型。

8.5 Complex Task Management

复杂任务管理（Complex Task Management）要求 Agent 把长目标分解成可执行子任务，跟踪进度，在每个阶段验证结果，并在失败后调整计划。Harness 可以提供计划模式、任务列表、分解工具或 Sub-agent；模型训练可以使用长时程任务，让 Policy 学会何时计划、何时执行和何时回退。

课堂提到一个很有意思的工程事实：某些“Plan Mode”最初可能只是向 Prompt 增加一句“请先规划，不要执行”。即使只是 Prompt，它仍然改变了模型可采取的 Action 范围。更成熟的 Plan Mode 还会在 Harness 层禁用写操作，直到用户审批计划。

8.6 Environment Understanding

环境理解（Environment Understanding）不是通用语言能力的自动副产品。GUI Agent 要理解 Screenshot 与界面结构，金融 Agent 要解释时间序列，科学 Agent 可能要分析专业图像，Coding Agent 则要理解仓库约定、测试日志和依赖关系。

Harness 可以提供领域 Skills，把原始环境转换成更清晰的 Observation；训练端可以使用与真实环境形态一致的数据，或让模型直接在领域环境中进行 SFT/RL。课堂特别强调，模型不会自己“自然变好”，而是有人构造数据和训练环境，使它在目标 Observation 形态上变好。

8.7 Safety

Agent Safety 既来自模型，也来自系统。Harness 可以把执行放进 Sandbox、限制凭据、网络和文件系统权限、要求高风险动作审批，并监控整条 Trajectory；训练可以使用 Safety-aware RL，提高模型主动拒绝越权或危险动作的概率。

只训练模型遵守规则不够，因为任何概率模型都可能犯错；只做 Sandbox 也不够，因为合法权限内部仍可能出现错误操作。稳健系统需要最小权限、隔离、审批、监控和模型行为训练共同工作。

9. 两条改进路线：训练 Model，还是工程化 Harness

9.1 LLM Training 改变 Learned Policy

模型训练直接改变参数中的行为倾向。课程把训练过程概括为三个阶段：

```
Pretraining
文本、代码、多模态数据
学习广泛表示与规律
        │
        ▼
Mid-training / SFT
指令、轨迹、工具调用
学习格式与示范行为
        │
        ▼
Reinforcement Learning
奖励或偏好
优化整条 Trajectory 的行为
```

Pretraining 建立基础表示；SFT 用正确示范教模型怎样按格式推理和调用工具；RL 则根据任务结果或偏好调整长时程 Policy。Agent 训练尤其困难，因为奖励可能只在许多步骤之后出现，中途 Action 还会改变后续可见状态。

9.2 Harness Engineering 改变模型周围的系统

Harness Engineering 不改模型权重，而是改变 Prompt、Tools、Memory、Control Flow、Validation、Retries 和 Safety Boundaries。它通常迭代更快，也能为特定产品立即增加约束。

课堂给出的经验判断是：新问题往往先在 Harness 层被识别和缓解；当问题足够普遍、训练团队有了合适数据之后，一部分能力会被训练进模型。但这不是“最终所有 Harness 都会消失”。Context 的物理成本、访问权限、业务规则、外部工具和用户临时偏好仍需要系统处理。

> 【图：Lecture 1 Slide 18 —— Two ways to build agent capabilities】

图左侧是 Training，它把可复用的推理、工具使用和恢复模式写入 Learned Policy；右侧是 Harness，它在推理时提供 Context、验证、重试与安全边界。中间的“Model + Harness”表示实际能力来自组合，而不是二选一。

9.3 同一个问题可以有两种解法

能力问题	Harness 路线	Training 路线
工具调用不合法	Schema 校验、约束解码	用工具轨迹做 SFT
长历史遗忘	压缩、Memory、Sub-agent	长上下文训练
不符合用户习惯	Prompt、Skills、Custom Tools	从用户反馈中学习
不会规划	Plan/Decomposition Tools	长时程任务 RL
看不懂环境	结构化 Observation、领域 Skill	领域 SFT/RL
行为不安全	Sandbox、权限、审批、监控	Safety-aware RL

工程决策要同时考虑时间、成本和泛化。训练可能更根本，但需要数据、算力和训练基础设施；Harness 能快速部署，却可能增加复杂度，或者只对当前环境有效。较小模型通常需要更多外部结构与 Guardrails，较强模型虽然泛化更好，也仍然不能绕过权限与可验证性。

10. Agents are systems, not just models

10.1 Model 只是 Agent System 的一部分

Lecture 1 最重要的结论之一是：**Agent 是系统，而不只是模型。** 一个真实 Agent System 更接近：

```
Agent System
├── Model
│   └── 根据 Context 生成 Reasoning 与 Action Tokens
├── Harness
│   ├── Prompt / Context
│   ├── State / History
│   ├── Tools
│   ├── Memory
│   ├── Validation
│   └── Control Flow
├── Sandbox
│   └── 隔离执行并限制资源、网络、文件与凭据
├── LM Inference
│   └── 服务、批处理、流式输出与 KV Cache
├── Training System
│   └── 数据、Rollout、Reward、分布式 Worker 与更新
└── Observability
    └── Trace、质量、延迟、成本与失败分析
```

Model 负责开放式判断，Harness 把判断变成受控程序，Sandbox 限制动作后果，Inference System 稳定提供生成，Training System 改进 Policy，Observability 则让开发者知道系统究竟做了什么。

> 【图：Lecture 1 Slide 26 —— Agents are systems, not just models】

图中心的 Harness 位于 Sandbox 内部，和外部 Model 通过 Inference 交互；Monitoring 观察系统运行，Training 改变模型。箭头说明能力与反馈跨越多个组件。只提高模型 Benchmark 分数，不能自动解决工具权限、执行隔离、缓存、日志和线上监控。

10.2 Harness：把概率字符串接到确定性软件

智能体执行框架（Agent Harness）是模型外部的控制程序。它管理 State、Tools、Memory 和 Control Flow，校验动作、处理错误，并执行权限与安全规则。

Harness 的价值在于把模型的不确定输出包进确定性协议。模型可以提出任何字符串，Harness 只接受合法工具、合法参数和获授权动作；工具失败时，它把错误转成可恢复 Observation，而不是让整个系统崩溃；模型达到 Step Limit 时，Harness 强制停止。

10.3 Sandbox：限制错误动作的最大影响

沙箱（Sandbox）把代码和工具执行隔离在受控环境中。它限制 CPU、内存、运行时间、网络、文件路径与凭据，并提供可复现初始状态。

在 SWE-bench 中，每个问题都从固定仓库状态开始，Agent 在 Sandbox 中修改代码；评测可以重新创建同样环境并回放补丁。Sandbox 既是安全设施，也是实验可复现设施。

10.4 LM Inference：Agent 对服务系统提出了更高要求

Agent 会频繁调用模型，History 还会越变越长。Inference System 需要可靠服务生成、批处理多个 Agent 请求、复用 KV Cache，并管理 Streaming、Parallelism 和 Throughput。普通聊天偶尔发送一次 Prompt，Agent 则可能几十或上百次重复携带大段相同前缀，因此缓存格外重要。

这部分对应 Lecture 3 的 Prefill、Decode、KV Cache 和 Context Compaction。Agent 系统问题不能只在 Python 循环层解决，底层推理效率会直接决定延迟和成本。

10.5 Training System：从轨迹中改进 Agent

训练系统准备数据、收集 Rollout、协调分布式 Worker、计算 Reward、更新模型，并对 Checkpoint 做评测和复现。对 Agent 而言，训练样本不是孤立问答，而可能是包含多轮 Action 与 Observation 的 Trajectory。

如果奖励只看最终结果，系统还要面对 Credit Assignment：成功或失败究竟由哪一步导致？如果环境不可复现，同一个 Policy 的结果也难以比较。因此，Training 与 Environment、Evaluation、Sandbox 紧密相连。

10.6 Observability：没有 Trace 就无法调试 Agent

可观测性（Observability）记录 Trajectory 与系统指标，包括工具调用、Observation、质量、延迟、Token、费用和错误。Agent 失败往往不是单一 Exception，而是早期误解引发的一连串错误决策；只有完整 Trace 才能找到起点。

Monitoring 还承担安全职责。高风险动作、异常网络访问、重复失败和成本突然上升，都需要被检测。最终结果看似正确，也不能证明过程中没有越权或偶然成功。

11. 普通 LLM、Chatbot、Workflow 与 Agent 的区别

11.1 用同一个任务观察结构差异

假设用户要求：“找到符合预算和时间要求的航班，并在付款前让我确认。”

普通 LLM 可以解释搜索策略，或者根据 Prompt 中已有信息推荐方案，但它不能凭文字生成获得实时航班。一次调用通常在输出建议后结束。

接入航班查询 Tool 的系统可以获得实时数据。如果程序固定调用一次搜索 API，再把结果交给模型总结，它具备 Tool Use，却仍然可能只是一个预先编排的工作流（Workflow）。

Agent 会把任务作为持续目标：根据用户约束决定第一次搜索参数，观察结果后发现没有合适航班，再调整日期或机场；比较候选项；到达需要付款的高风险节点时，根据 Harness Policy 请求用户审批；用户确认后才调用预订工具，最后检查订单状态。每一步都依赖前一步 Observation。

结构性差异可以概括为：

系统	核心形态	是否必然多步	谁决定下一步	是否能改变外部环境
LLM	Context → Token Sequence	否	一次生成过程	模型本身不能
Chatbot	围绕对话管理 LLM 调用	可能	用户、固定逻辑或模型	取决于是否接入工具
Tool-enabled Application	模型可提出或使用工具调用	可能	固定逻辑或模型	可以，由外部执行器完成
Workflow	预先定义节点与转移	通常是	程序规则为主	可以
Agent	在环境反馈中持续选择 Action	通常是	模型 Policy 与 Harness 共同决定	可以

这些词在产业和论文中没有绝对统一的边界。一个 Chatbot 产品可以内置 Agent，一个 Workflow 的某个节点也可以由 Agent 控制。课程真正强调的是系统结构：是否存在环境、Observation、Action，以及模型是否在循环中根据新状态选择后续行为。

11.2 “只要能调用 Tool 就算 Agent 吗？”

不一定。若模型只在一次请求中调用固定工具，然后立即返回结果，它具备 Tool Calling，但未必体现长时程自主决策。Tool 是连接环境的 Action 接口；Agent-like Behavior 来自工具、反馈历史和循环控制的组合。

另一方面，Agent 也不一定使用标准 JSON Function Calling。mini-swe-agent 只暴露 Bash，模型可以直接生成 Shell Command；GUI Agent 可能生成鼠标坐标；机器人可能输出控制信号。判断重点不是具体协议，而是这些输出是否被解释为 Action，并在环境反馈后继续决策。

11.3 “ChatGPT 本身是不是 Agent？”

不能脱离具体配置回答。底层语言模型只是生成模型；聊天产品可能只管理多轮文本，也可能接入搜索、代码执行、浏览器、文件和任务循环。把整个产品称为 Agent 是否合适，取决于它实际开放了哪些 Action、怎样处理 Observation，以及是否能自主推进多步任务。

因此，“某模型是不是 Agent”通常问错了层级。更准确的问题是：“围绕这个模型构建的系统，是否形成了感知—行动—反馈循环？”

11.4 Agent 与 Workflow 的边界

Workflow 强调开发者预先定义流程，例如“先检索，再总结，再发邮件”；Agent 强调模型根据当前状态选择下一步。Workflow 的可预测性和可审计性更强，Agent 的 Action Space 和适应性更大。

现实系统经常混合二者：固定 Workflow 控制关键业务阶段，每个阶段内部由 Agent 处理开放任务；或者 Agent 提出计划，Workflow 要求人工审批后才允许进入写操作。越高风险的任务，越有理由把自由决策限制在清晰边界内。

12. 11-768 后续会研究什么

12.1 从一个最小循环自然展开

最小 Agent 只需要模型、工具、历史和循环，但它马上会暴露一连串问题：

```
Minimal Agent Loop
        │
        ├── 调用经常出错 ──────► Tool Use
        ├── History 不断增长 ──► Context Management
        ├── 跨任务需要保留信息 ─► Memory / Skills
        ├── 长任务容易迷失 ─────► Planning / Delegation
        ├── 不理解特定界面 ─────► Coding / GUI / Domain Agents
        ├── 不知道是否真的成功 ─► Evaluation
        ├── 行为模式不稳定 ─────► SFT / Reinforcement Learning
        ├── 动作可能造成伤害 ──► Sandbox / Safety
        └── 与人或其他 Agent 协作 ► Interaction / Multi-agent Systems
```

Lecture 2 研究 Tool Definition、Tool Call、约束解码和工具接口；Lecture 3 研究长 History 对推理、缓存和压缩的影响。Memory 与 Skills 处理活动 Context 之外的长期信息，Planning 处理目标分解和进度控制。

在能力基础之后，课程进入 Coding Agent 与 GUI Agent 等应用域，再讨论 SFT、RL 和训练系统；随后研究 Framework、Sandbox 与 Safety，最后扩展到 Human-Agent Interaction、Multi-Agent System 和 Search-based Reasoning。课程路线不是互不相关的主题列表，而是围绕同一个 Agent Loop 逐层增强。

> 【图：Lecture 1 Slide 34 —— Semester schedule】

图按时间依次排列 Agent Capabilities、Domains、Training、Frameworks and Safety、Interaction and Projects、Advanced Topics。前半段建立“能运行、能评测、能训练”的基础，后半段把这些能力放进具体系统与研究问题。

12.2 课程希望学生获得什么能力

课程的学习目标包括从开源 LLM 之上实现 Agent、为多步任务设计 Evaluation、训练 Agent 改进能力、分析 Safety 与 Reliability Trade-off，并围绕 Agent 开展一个开放研究问题。

三次个人 Assignment 分别聚焦 Harness、Evaluation 和 Training，后半学期用团队 Project 把任务、环境、评测与研究问题组合起来。Lecture 1 的作用，是让后续实现始终回到同一系统视角，而不是把“换模型”“加 Prompt”“接工具”当作彼此孤立的技巧。

课程正式先修要求是有较扎实的语言模型训练经验。这篇笔记可以帮助建立 Agent 概念，但后续 Training Assignment 仍要求学生能处理 SFT、RL 和分布式训练相关实现。

13. 与 Assignment 1 的关系

Assignment 1 的标题是 “Build an Agent Harness”。Part 1 要在共享 **Agent** 类中实现领域无关的 ReAct Loop，再由 **CodeAgent** 提供终端工具，在 Sandbox 中修复一个 Chess App 的软件问题。

Lecture 1 的概念与代码一一对应：

Lecture 1 概念	**Assignment 1 中的体现**
Context	build_prompt() 生成的 Message Sequence
Task	User Role 中的软件问题
History / State	先前 Assistant Action 与 Tool Observation
Model	query_language_model() 调用的 OpenAI-compatible LLM
Action	execute 、 **send_message** 、 **invoke_skill** Tool Call
Observation	带 **role="tool"** 和 **tool_call_id** 的执行结果
Environment	远程 Modal Sandbox 与 /testbed 仓库
Agent Loop	构造请求、取得 Action、执行、追加 Observation、重复
Stop Condition	**send_message** 、 **finished** 、 **step_limit**
Trajectory	保存的 Prompt、Response、Tool Result 与 Compaction Event

Part 1 首先要求维护 Agent State，并把 System Instruction、Task、历史推理、Action 与 Observation 组织成 Prompt。随后实现 Agent.run ： 每一轮请求模型、提取 Tool Calls、执行工具并把结果放回下一轮。

CodeAgent 的 **execute_tool_calls** 必须处理多个调用、未知工具和格式错误。错误不能简单抛出并终止整个任务，而应转成模型能够观察和修正的 Tool Message。这就是“错误可恢复”的 Agent Loop。

作业还用 Skill 教 Agent 如何提交 **patch**.txt 。System Prompt 只暴露 Skill 名称与描述，模型需要时再调用 **invoke_skill** 读取完整内容。这是 Progressive Disclosure：Context 中先放少量索引，细节按需进入，避免一次塞入所有知识。

最终运行会产生 Patch 和 Trajectory；评测把 Patch 应用到干净环境并运行测试。学生因此同时接触 Lecture 1 的 Harness、Environment、Action、Observation、Trajectory、Sandbox 与 Reward/Verifier，而不是只写一个模型 API 调用。

14. 容易混淆的概念

14.1 Model、Harness、Tool 与 Environment

概念	它负责什么	它不负责什么
Model	根据 Context 生成 Token、Reasoning 和 Action Proposal	不直接执行操作系统动作
Harness	管理 Context、循环、验证、权限、错误与停止	不代替模型完成开放式判断
Tool	定义某类可调用 Action 的接口	定义不等于已经执行
Environment	保存真实状态并执行动作	不负责决定总体目标

当模型输出 Tool Call 时，Harness 才是调用方，Environment 才是动作发生的位置。把四者混在一起，会让安全分析变得含糊：模型生成了危险建议、Harness 错误授权、还是 Tool 实现越权，属于不同故障。

14.2 State、Observation、Context、History 与 Trajectory

概念	观察角度
State	环境此刻真实处于什么情况
Observation	Agent 这一步获得了哪些关于 State 的信息
Context	这一轮真正输入模型的 Token 或多模态内容
History	已发生的消息、Action 与 Observation 记录
Trajectory	围绕一个任务展开的完整多步执行路径

Harness 通常从 History 中选择和格式化内容来构造 Context；Context 不一定包含全部 History。Observation 也不一定等于完整 State。Lecture 3 的 Compaction 和 Lecture 4 的 Memory，正是在管理这些集合之间的关系。

14.3 Observation 与 Reward

Observation 描述后果，Reward 评价后果。 **pytest** 输出了两个失败测试，是 Observation；根据是否全部通过给 0 或 1，是 Reward。LLM-as-a-Judge 的评语也可以同时包含文字 Observation 与数值 Score，但功能仍应区分。

14.4 Reasoning、Planning 与 Acting

Reasoning 是模型为当前判断生成的中间推理；Planning 更强调对未来多个步骤的组织；Acting 是把选择变成环境操作。ReAct 允许它们交替发生：先根据当前证据做短程推理并行动，得到新证据后再修正计划，而不是一开始生成永不改变的完整方案。

14.5 Tool Calling 与 Tool Execution

Tool Calling 是模型输出一种协议表示；Tool Execution 是外部程序真正运行函数。前者可以通过训练与约束解码改进，后者必须由权限检查、Sandbox、超时和错误处理保护。生成了调用不代表调用成功，成功执行也不代表它符合用户意图。

总结

1. 经典 Agent 的共同结构是感知环境并作用于环境；11-768 主要研究以 LLM 为决策核心的多步 Agent System。
2. Environment 保存外部状态，Observation 是 Agent 得到的局部信息，Action 是对环境施加的操作，Reward 用于评价结果质量。
3. LLM 本身只处理 Token。Tool Call 也是一种 Token Sequence，真正的解析、验证和执行由 Harness 完成。
4. Agent 与一次普通 LLM 调用最大的结构性区别，是 Action 改变环境、Observation 回到 Context、模型再决定下一步的闭环。
5. Tool Use 是 Agent 行为的接口基础，但单次工具调用不自动等于完整 Agent；History、反馈和持续控制同样重要。
6. Agent 的自主性表现为模型能在多大范围内选择工具、参数、顺序、重试和停止；自主性越高，权限与审批设计越重要。
7. 一个可用 Agent 需要准确工具调用、长上下文连贯、可定制、复杂任务管理、环境理解和安全六类能力。
8. Agent 能力可以通过 LLM Training 固化进 Learned Policy，也可以通过 Harness Engineering 在推理时提供结构，两条路线通常共同使用。
9. Model 只是 Agent System 的一部分；Harness、Sandbox、Inference、Training 和 Observability 决定动作能否安全、可靠、可复现地发生。
10. 后续 Tool、Context、Memory、Planning、Evaluation、Training 和 Safety 主题，都可以看成在完善 Lecture 1 的基本 Agent Loop。

```
Lecture 1: Course Overview — What Is an Agent?
├── Foundation
│   └── Agent = Perceive Environment + Act on Environment
├── Components
│   ├── Environment / State
│   ├── Observation
│   ├── Action
│   └── Reward
├── LLM-to-Agent Bridge
│   ├── Tool Definition
│   ├── Action as Tokens
│   └── Harness Execution
├── ReAct Loop
│   └── Context → Reason → Act → Observe → Repeat / Stop
├── Capability Building
│   ├── LLM Training
│   └── Harness Engineering
└── System Stack
    ├── Model
    ├── Harness
    ├── Sandbox
    ├── Inference
```

```
    ├── Training
    └── Observability
```

References

1. CMU 11-768: AI Agents — Course Website and Syllabus
2. Lecture 1 Slides — What Are Agents? And How Do They Work?
3. Lecture 1 Recording — CMU AI Agents 2026
4. Assignment 1: Build an Agent Harness — GitHub Repository
5. Assignment 1 Instructions
6. Toolformer: Language Models Can Teach Themselves to Use Tools
7. ReAct: Synergizing Reasoning and Acting in Language Models
8. mini-swe-agent — Official GitHub Repository
9. mini-swe-agent Default Agent Loop Source
10. Building a C Compiler with a Team of Parallel Claudes — Anthropic Engineering
11. Claude’s C Compiler — Official GitHub Repository
12. SWE-bench — Official Website
13. 课程学习笔记的组织形式参考

人工智能　　# 语言模型